# Parallelizing Quantized LLM Inference Under a Fixed Numerical Reference

Chengzhan Li[a,*]

[a]University of Electronic Science and Technology of China, Chengdu, China

ARTICLE INFO

*Keywords*:
Large language model inference
Parallel execution
Quantization
Numerical reproducibility
KV cache
Speculative decoding

ABSTRACT

Batching known queries can change a quantized language model's outputs and persistent state even when the causal token dependencies are unchanged. We study parallel execution that preserves a fixed deployment's numerical reference. The method separates two grouping decisions: the physical KV range exposed to attention and the arithmetic organization used by quantized projections. It places required range boundaries locally inside attention, exposes independent arithmetic groups to the backend, and preserves the outputs and state needed by downstream consumers. A conditional composition argument connects these obligations across decoder calls. Implementations in CUDA and Vulkan show where this construction creates useful parallel work. On an A5000 with Qwen3-32B, independent scalar-column groups reduce target-only resident-service time by 1.51% relative to a strong backend scalar-loop control (95% paired-root interval: 1.23–1.79%). Q8 factorial experiments separate projection packing from local-cut effects: packing contributes 0.80–1.64%, while local cuts add little on the tested natural requests. Controlled boundary crossings benefit substantially, but a two-conversation multi-turn pilot shows no local-cut service improvement. Complete speculative applications improve on their target-only controls; their larger gains are distinct from the execution method's increment. A modern upstream operator is faster under a different numerical contract. These results identify a practical compatibility option: preserve an established reference while reorganizing the parallel work that its arithmetic and state requirements still permit.

## 1. Introduction

Suppose a decoder is about to evaluate four known token positions. Processing them together can reuse weights and expose more GPU work. Yet crossing a KV-cache padding boundary can enlarge the physical attention range seen by earlier queries, even when the causal mask hides the same future tokens. The resulting kernel can produce different output bytes. A quantized projection can introduce another difference when query width selects a different quantizer or dot-product specialization.

Such differences matter when an optimization must retain the result of an existing deployment. For example, a regression workflow may compare full output vectors, an exact replay may resume from a stored cache, and a component migration may exchange states with an unchanged consumer. These scenarios motivate our question: *what parallel execution remains available when the numerical reference is fixed?*

Existing work provides several answers to related problems. Batch-invariant kernels make results insensitive to batch composition [1]; deterministic scheduling can keep selected execution shapes fixed [2]. Reconstruction of GPU arithmetic can reproduce an external matrix-multiplication implementation while changing scheduling and pipelining [3]. Verified speculation can obtain deterministic inference by checking and repairing a fast path, including its KV state [4]. These approaches establish that reproducibility is a system property and that the choice of reference matters. Our focus is a composition problem in quantized decoding: preserve an established numerical path while changing how known queries share physical cache views and backend execution.

The central insight is that physical grouping and arithmetic grouping answer different questions. A *physical group* determines which queries can share the attention range and descriptor required by their reference. An *arithmetic group* retains the column width, activation quantization and reduction organization of a qualified projection. Their boundaries need not coincide. We keep the surrounding decoder graph wide, split attention where the physical reference requires it, and expose independent arithmetic groups through an outer tensor dimension. Requested output rows receive owned storage, and accepted state is committed with the same logical frontier. Figure 1 presents this construction.

The construction combines established four-column kernels, weight broadcasting, packing, and fixed reduction orders around two distinct grouping constraints. Its method contribution lies in the placement of cuts inside the decoder and the output and state obligations that make the result usable by a downstream decoder. This yields an implementation recipe with explicit admission conditions and a conditional composition argument. It also makes the performance question

*Corresponding author.
*Email address:* chengzhan.li@std.uestc.edu.cn (C. Li)

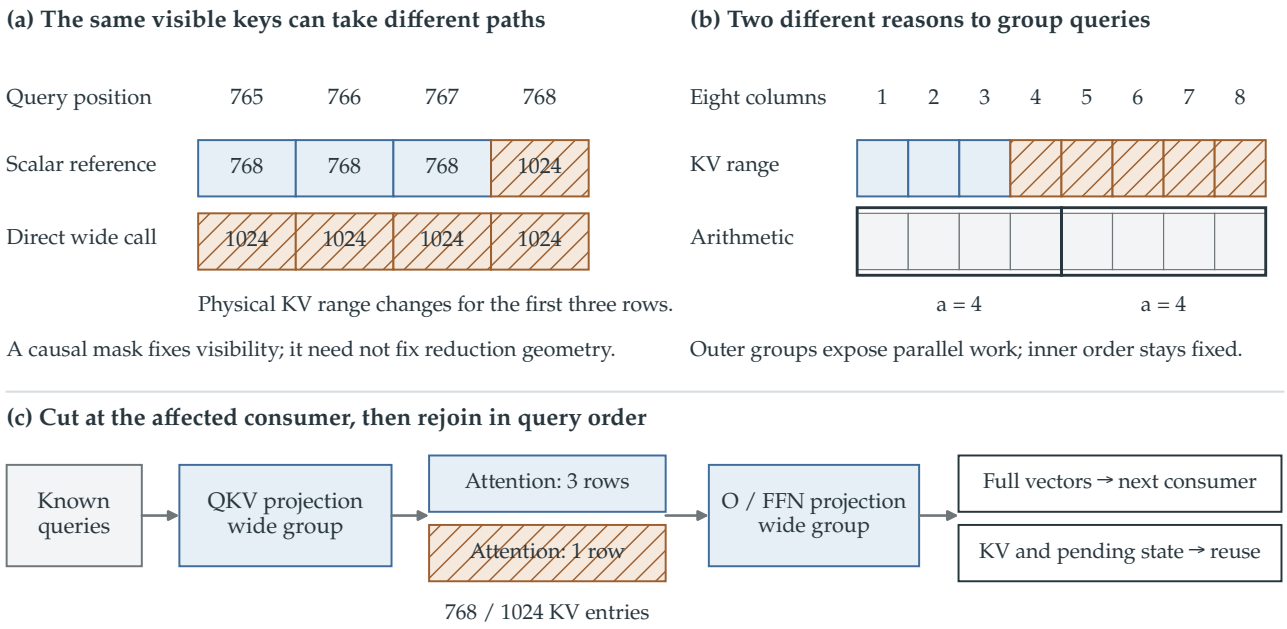



**Figure 1:** Parallel work around a fixed numerical reference. Direct batching can enlarge a query's physical attention range. The construction instead retains the reference ranges inside attention, preserves the inner arithmetic of projection groups, and reconstructs ordered outputs and successor state for the consumer. Colored boundaries and line patterns distinguish physical ranges from arithmetic groups; the drawing describes execution dependencies, not concurrent execution of causally unknown tokens. Conceptual diagram developed with ChatGPT (OpenAI, GPT-6 Astra Pro) assistance and rendered from the supplied editable script.

testable: a local cut helps only when the wider surrounding work it preserves pays for its own overhead.

We evaluate that question with interventions, strong controls and application measurements. A physical-range intervention reproduces the observed attention-output change while holding the visible operands fixed. On A5000, a target-only Qwen3-32B comparison isolates independent scalar-column execution from a backend control that already shares activation quantization. The additional resident-service reduction is 1.51%, with a 95% paired-root interval of 1.23–1.79%. Balanced Q8 factorials show that projection packing and local attention cuts contribute at different scales. Controlled Vulkan crossings reveal a substantial local opportunity, whereas natural requests encounter few useful crossings and a small multi-turn pilot shows no service gain. Speculative applications provide a consumer of the compatible execution path, with their complete gains reported separately from the method's increment.

The paper makes three contributions:

1. A fixed-reference execution construction that separates physical KV grouping from arithmetic grouping, places required cuts locally, and preserves the consumer's requested outputs and successor state.
2. An implementation analysis that connects that construction to quantized CUDA and Vulkan paths, distinguishing run-time checks, offline qualification and the assumptions of the composition argument.
3. An evidence-based account of the available parallelism: strong same-contract controls, component attribution, complete application measurements, and counterexamples that explain when local cuts or wider speculation fail to improve service.

The resulting claim is a compatibility method with measured benefits in defined deployments. Its value depends on retaining the reference: changing the numerical contract opens other choices, including a faster modern upstream operator examined in our evaluation.

## 2. Parallel execution with a fixed reference

### 2.1. Two grouping decisions

Consider four known queries at positions 765–768. A scalar continuation path with a 256-entry KV padding unit exposes physical attention ranges of $[768, 768, 768, 1024]$. Executing the queries together can expose 1024 entries to every query. The causal mask still hides the same future tokens, yet the attention kernel now receives a different physical problem. In the captured CUDA case, the visible operands and probability prefixes agree, and the additional probabilities are zero; changing the physical range alone reproduces the two different attention–value outputs [5]. A mask preserves mathematical visibility, but need not preserve the executed reduction.

Splitting the entire decoder into calls of three queries and one query restores the required ranges. It also splits projections and feed-forward work that need no such boundary. Our construction keeps the wider decoder graph and places the split inside attention. This is the first grouping decision: which queries can share a physical KV range? The second concerns arithmetic: which query columns can share an existing quantizer and reduction specialization? These groups need not coincide (Fig. 1).

For example, a supported CUDA Q8 projection preserves the established four-column arithmetic while presenting several such groups as independent channels. The mixed Q4_K/Q6_K implementation instead retains one-column arithmetic. Neither choice follows from the weight format alone: activation representation, reduction organization, tensor strides and device placement also determine the result. Parallel work is exposed around those decisions rather than by changing them.

## 2.2. Reference and consumer contract

The reference is a particular deployment, denoted $R_d$. It fixes model and tokenizer bytes, input formatting, numerical modules, placement, cache types and capacity, prompt production, requested outputs, and sampling and stopping rules. A different device or numerical backend defines a different reference.

Let $O$ be the output rows requested by the consumer. For a materialized prefix $x$, let $z_i(x)$ be row $i$'s complete binary32 vocabulary vector, and let $K(x)$ contain active keys and values in canonical logical order. Let $s(x)$ record logical positions, the retained frontier, any emitted but unmaterialized token, and termination status. The compatibility target at the relevant boundaries is

$$\begin{aligned} z_i^F(x) &\overset{\text{bits}}{=} z_i^{R_d}(x), \quad i \in O, \\ K^F(x) &\overset{\text{bits}}{=} K^{R_d}(x), \qquad s^F(x) = s^{R_d}(x). \end{aligned} \tag{1}$$

Unused cache storage is outside this relation. Token agreement is a consequence for the specified greedy consumer, not a substitute for the vector and state obligations. A vector omitted from $O$ must be excluded by the consumer before execution, rather than discarded after finding a discrepancy.

Speculative verification illustrates why this distinction matters. The target evaluates a seed and draft proposals, including prefixes that may later be rejected. Their requested vectors are compared with an independent reference following those same prefixes after the fixed prompt producer. A separate autonomous reference supplies the generated-token endpoint. This preserves the target's obligations without requiring the draft to reproduce the reference trajectory.

## 2.3. Constructing compatible groups

The construction takes an ordered sequence of known token rows, their positions and requested-output flags, the incoming cache, and backend-specific numerical requirements. It returns an execution organization and owned outputs in the original logical order. Backend analysis supplies the numerical requirements; the planner does not discover equivalent kernels automatically.

For each row $i$, determine the physical extent $e_i$ that its reference call would expose. In the admitted dense, nonwrapping cache, with padding unit $P$, capacity $C$, and zero-based position $p_i$, this is

$$e_i = \min\left(C, \max\left(P, P\left\lceil \frac{p_i + 1}{P} \right\rceil\right)\right). \tag{2}$$

The implementation obtains the padding and cache metadata from the native engine and checks the append mapping. It retains producer-specific behavior when a prompt phase uses a different reference schedule.

A *physical group* is an adjacent run with the same extent and compatible attention descriptor, including relevant data types, layouts and dispatch conditions. Rows remain in their original order. Each run is tiled to the admitted attention width. A whole-call implementation, SIMPLE, executes these groups as separate decoder calls. The LOCAL implementation creates matching Q, K, V and mask views inside attention, computes each tile, and concatenates its outputs in logical order. QKV projections and feed-forward operations remain in the enclosing wider graph. This changes where the boundary is paid, not the causal dependencies between tokens.

An *arithmetic group* retains a qualified column width $a$ and its numerical specialization. With an activation matrix $X \in \mathbb{R}^{K \times q}$, separate complete groups from the tail before taking the grouped view:

$$\begin{aligned} X[K, q] &= [X_{\text{full}}[K, ag]\ X_{\text{tail}}[K, r]], \\ X_{\text{full}}[K, ag] &\longrightarrow X_{\text{full}}[K, a, g], \\ q &= ag + r, \qquad 0 \le r < a. \end{aligned} \tag{3}$$

The outer dimension exposes independent groups to the existing backend while the inner width remains $a$; a tail uses an admitted smaller width. The weight tensor is broadcast across groups. Preserving a shape alone is insufficient: the selected kernel must retain the required activation quantization, inner reduction, strides and placement. Each deployment binds these choices to a fixed implementation; qualification checks the resulting outputs and state for the reported cases.

The reusable construction therefore has four steps:

1. Bind the producer, incoming state, consumer outputs and backend numerical behavior.
2. Derive each row's physical extent and partition the attention work at changes in extent or numerical descriptor.
3. Retain the admitted inner arithmetic while exposing independent outer groups; keep extent-independent work outside local attention cuts.

4. Copy requested outputs into row-owned storage, then commit the accepted state and logical frontier together.

Equal-descriptor runs minimize the number of adjacent groups under those equality and width constraints. They do not minimize end-to-end latency: local views, launches, concatenation and output ownership all incur costs. The measured comparison between SIMPLE and LOCAL determines whether retaining wider work repays those costs.

### 2.4. Why the construction composes

The argument is conditional on three obligations: the incoming producer state satisfies Eq. 1; each admitted operator organization preserves its reference values and state effects; and the consumer receives the required outputs with the same commit semantics. Under these conditions, induction over the dependency graph preserves each intermediate required by subsequent operations. Physical views retain the reference operands and arithmetic, row-wise reconstruction retains output identity, and state updates retain the active prefix. Applying the same argument at successive commit boundaries preserves continuation.

Output ownership is essential to the last step. A later native call can reuse an earlier output buffer, while speculative acceptance can select an earlier row as the retained frontier. The executor therefore copies every requested vector before another group can overwrite its storage. It trims rejected KV entries and records a correction or bonus token as pending until that token is actually materialized. A generation limit or end-of-generation token can stop execution in this pending state.

This argument explains the obligations that an implementation must discharge. The experiments test the contract at their reported cases and boundaries; the argument is not an all-input proof of the native kernels. The methodological contribution is the separation of physical and arithmetic grouping, local placement of the resulting cuts, and composition through an explicit consumer contract. Packing, fixed reductions, weight broadcasting and existing small-column kernels remain reusable components of the construction [6].

## 3. Implementation

### 3.1. Backend organizations and fair controls

We implement the construction in a modified llama.cpp execution path [5]. The phase executor retains a resident target context and binds the method to a dense, contiguous, nonwrapping cache with one owned sequence and F16 keys and values. Its CUDA attention path uses causal, non-Flash attention for the supported Llama and Qwen architectures. The backend-independent planner carries ordered rows, physical extents and requested-output flags; backend code supplies the actual views, numerical specializations and placement decisions.

The Q8 implementation preserves the established small-column arithmetic [6]. Its packed projection presents complete four-column groups as channels and reconstructs the original column order. The strong *backend-loop4* control quantizes the wide activation once, then invokes the existing four-column and tail kernels in a backend loop. This removes repeated graph-level quantization as an explanation for any remaining packed-path gain. It also distinguishes the cost of graph slicing from the benefit of exposing independent groups to the backend. With three draft proposals, verification has four rows; multi-group Q8 packing therefore arises in wider prompt work in those experiments.

The 32B mixed Q4_K/Q6_K experiment uses a different arithmetic unit: one column. Its strong control, B, quantizes the activation once and loops over scalar backend projections. Candidate C uses a $[K, 1, q]$ activation view to expose those scalar computations as independent channels, followed by output reshaping. B is therefore a backend scalar-loop control, not backend-loop4. Both arms retain the same quantized weights and one-column dot-product specialization. Their comparison measures the additional effect of execution organization beyond shared activation quantization.

The Vulkan companion preserves F32 activations where the scalar reference uses F32 and wider native dispatch would select Q8_1. Its attention implementation also retains the relevant physical extent and layout. These backend-specific choices implement a common construction; they do not imply equal results between CUDA and Vulkan.

### 3.2. Resident execution and graph reuse

SIMPLE issues decoder calls at the planned boundaries. LOCAL retains wider phase calls and builds the cuts within attention, using the native cache's per-row extent vector. Both preserve the caller's output requests. After synchronizing each call, the executor copies requested full vocabulary vectors into owned storage and checks the physical append endpoint. The clean path performs neither reference replay nor state export to construct its plan.

Graph reuse accounts for more than outer tensor shape. The live implementation compares the numerical policy, the complete physical-extent vector, relevant head-normalization and rotary-placement decisions, and the engine's normal topology and output-count conditions. A mismatch rebuilds the graph. The deployment freezes additional model and module choices; the small frontend arithmetic tag, which encodes cache types, is not a complete numerical fingerprint.

### 3.3. Checks, qualification and failure behavior

Three mechanisms have distinct roles. Online checks admit the dense cache, supported widths and

requested-output reserve; marked backend-loop4 operators additionally check tensor descriptors, actual device residency and dispatch constraints. Capture runs observe selected operators and collect vectors and native state for offline comparison. The conditional argument in Section 2 then relates those observations to the consumer contract. Clean service timings exclude the diagnostic captures.

The packed Q8 nodes use ordinary backend dispatch and do not carry the loop4 marker, so they do not inherit all of that control's device and dispatch checks. Their evidence is deployment-specific qualification. Likewise, placement observation is enabled during capture rather than on every clean call. These differences are recorded with the exact implementations in the supplement.

Exceptions during target-group execution or its postchecks poison the target context, preventing its reuse. Other failures, including trimming, draft execution and sampling, are contained by termination of the published command-line worker. Some unsupported graph conditions use assertions. This is the implemented failure boundary; a persistent service that catches errors and continues would require transaction-wide invalidation before reusing its state. No automatic replay into a partially updated owner is performed.

## 4. Evaluation design

### 4.1. Questions, deployments and controls

The evaluation asks four questions. Does changing the physical attention range explain an observed numerical discrepancy? Does compatible grouping improve on a control that already shares quantization? When does placing the cut locally repay its cost? How does the execution increment relate to the cost of a complete speculative application?

The implementations run on an RTX A5000 with Linux/CUDA, an RTX 5070 Ti Laptop GPU with Windows/CUDA, and an Intel Arc B570 with Vulkan. Each deployment has its own reference. The common admitted profile is a single dense, nonwrapping sequence, F16 keys and values, greedy generation, and non-Flash causal attention. Natural-request studies use a context capacity of 1024; controlled Vulkan appends also use 4096. Models come from the Qwen3, Llama and SmolLM2 families [7–9]. Table 1 summarizes the panels supporting the main conclusions. Supplementary material provides frozen source locations, qualification coverage, secondary measurements and all plotted inputs [5].

In the Q8 panels, W and L denote SIMPLE whole-graph cuts and LOCAL attention cuts, while L4 and P denote backend-loop4 and packed projections. M denotes target-only generation, which loads no draft model. D4 denotes speculative generation with draft prefill microbatch four; the proposal count is $k = 3$. Thus, D4 is a draft setting, $q$ counts target query rows, and Q8_0 is a weight format. Qwen targets use a Qwen3-0.6B draft; the Llama pair is Meta-Llama-3.1-8B-Instruct with Llama-3.2-1B-Instruct. All arms receive the same applicable residency, graph reuse and numerical fixes.

The 32B panel isolates the method without a draft. It fixes whole-graph attention cuts, two CPU threads, context capacity 1024, target microbatch and output reserve 32, and disabled CUDA graphs. Its 12 new roots comprise six coding and six mathematics tasks, with 39–187 prompt tokens and a 128-token output cap. Each of 144 fresh workers performs an identical-root warmup followed by one formal request. G slices scalar projections at graph level; B shares activation quantization and loops over scalar backend calls; C exposes those scalar computations as channels. C/B is the primary method comparison.

The Qwen Q8 studies use pinned APPS and MATH inputs [10, 11], reusing task identities across sizes and platforms but establishing each deployment's reference separately. Llama's 12 roots were selected after excluding prior project exposure. The multi-turn pilot uses two development conversations from OpenAssistant [12]. These fixed workloads support deployment-specific estimates; model size is not a substitute for independent workload sampling.

### 4.2. Qualification and clocks

Offline qualification compares requested full vocabulary vectors and active KV entries in canonical logical order. Forced-prefix reference runs check speculative branches; an autonomous reference checks emitted tokens and stopping. Across qualification campaigns, the checked cases include prefill, continuation, boundary crossing, rejection, trim, pending tokens, termination and reset. Clean timing disables heavy observers and retains the corresponding endpoint, identity and execution-trace checks. The supplement associates observation scope with each implementation; it does not treat a clean endpoint check as a new full-state comparison.

Resident-service time includes reset, planning, target and draft work, sampling, state bookkeeping, synchronization and requested-output copies. It excludes model loading, context construction and diagnostic export. Worker clocks include initialization and primers and are reported separately. Neither clock represents a concurrent network service or a queueing-latency distribution. Prepared-state append measurements and extracted GPU-event operator timings have narrower boundaries, stated in their captions.

### 4.3. Estimands and uncertainty

For the 32B primary comparison, we average paired log time ratios over four rounds within each root and

**Table 1**
Design and independent units. A root is a task input, not an individual repeat. Repeated observations of a root remain together in resampling. The Qwen Q8 service and factorial panels share the same 16 roots; measurements from different panels are not pooled.

| Question / panel | Deployment | Comparison | Units and measurement boundary |
|---|---|---|---|
| Strong same-contract control | A5000; Qwen3-32B, Q4_K_M | Graph scalar G; backend scalar-loop B; independent channels C | 12 roots × 4 rounds; 144 measured and 144 warmup requests; resident service |
| Component attribution | A5000; Qwen3-8B and 14B, Q8_0 | SIMPLE/LOCAL × loop4/packed; draft fixed | 16 roots × 4 balanced rows per model; 256 measured and 32 primers; resident service |
| Complete application | A5000; Qwen3-8B, 14B and Llama-8B, Q8_0 | Four target-only controls; two speculative paths | 16/16/12 roots × 4 sessions; 384/384/288 measured requests; resident service and separate worker cost |
| Local-cut geometry | B570; SmolLM2-1.7B | SIMPLE versus LOCAL; crossing and inside | 12 fixed cells, 3 paired repeats; prepared-state append, excluding state preparation |
| Natural multi-turn pilot | A5000; Qwen3-8B, Q8_0 | SIMPLE versus LOCAL; packing fixed | 2 development episodes × 2 rounds; descriptive service comparison |
| Modern operator alternative | RTX laptop; Qwen3-8B, Q8_0 | Fixed-reference B versus upstream U | 1 captured tensor, 12 pairs per boundary; extracted operator with a different numerical contract |

then equally over 12 roots:

$$\rho_{C/B} = \exp\left(\frac{1}{12}\sum_{r=1}^{12}\frac{1}{4}\sum_{s=1}^{4}\log\frac{t_{rsC}}{t_{rsB}}\right). \tag{4}$$

The paired-root bootstrap resamples six coding roots and six mathematics roots within their strata, preserving their equal contribution. It uses 20,000 draws and a 95% percentile interval for C/B; the two secondary G comparisons use 97.5% intervals. All repetitions and arms of a sampled root remain together. The ratio of pooled totals is a different estimand and is identified separately.

A5000 Q8 service and factorial effects use the ratio of total service time, $\sum_{r,s} t_{rsA} / \sum_{r,s} t_{rsB}$. Their 10,000-draw paired-root bootstrap uses 99% intervals for the five-comparison family. The interaction is an equal-root geometric ratio of component ratios. The factorials use four-arm Williams orders; six-arm service orders have partial balance, and the earlier 8B speculative pair retains its fixed order. Intervals condition on the observed sessions and tested roots. Three-repeat structural cells show their paired ranges; the two-episode pilot has no population interval. All time reductions are $100(1-\rho)$, not throughput increases.

The accompanying CPU scripts reaggregate the published rows and reconstruct the figures. ChatGPT (OpenAI, GPT-6 Astra Pro, accessed 13 November 2026) assisted manuscript preparation, analysis and visualization code, and the conceptual diagram. Numerical plots are generated from identified measurements, not generated images. The prior research workflow also reports AI-assisted code development; its complete tool history and final disclosure require author verification. This preparation performed CPU reanalysis and document builds, with no new model execution.

## 5. Results

### 5.1. Physical range is part of the numerical problem

The CUDA diagnostic holds the query width at one and intervenes on the physical range supplied to attention–value multiplication. Cropping the wider execution from 1024 to 768 entries reproduces the shorter-reference output; padding the shorter execution to 1024 reproduces the wider output (Fig. 2). Visible operands and probability prefixes agree in the captured case, and the added probabilities are zero. The bidirectional intervention supports the physical-range mechanism at that operator. It observes the complete attention–value output, rather than inferring equality from the eventual argmax.

At its tested boundaries, the 32B strong-control qualification passes comparisons of full 151,936-entry vocabulary vectors and active F16 KV across 64 layers. These observations complement the construction's conditional argument. The supplement identifies the coverage and retained records for the other profiles. Its CPU counterexamples show why the output and state obligations are necessary: matching argmax values can coexist with changed full vectors, serialized state and forced continuation.

### 5.2. Independent groups improve on shared quantization

Figure 3 shows the strongest direct service comparison for the execution organization. On the 32B target-only deployment, C/B is 0.984864, with a 95% paired-root interval of [0.982148, 0.987721]. The corresponding reduction is 1.51% [1.23%, 1.79%]. All 48 measured paired requests favor C, but the inferential unit is the 12 roots, not the 48 repeats.

The weaker control explains why baseline choice matters. B/G is 0.956855 and C/G is 0.942372. Much

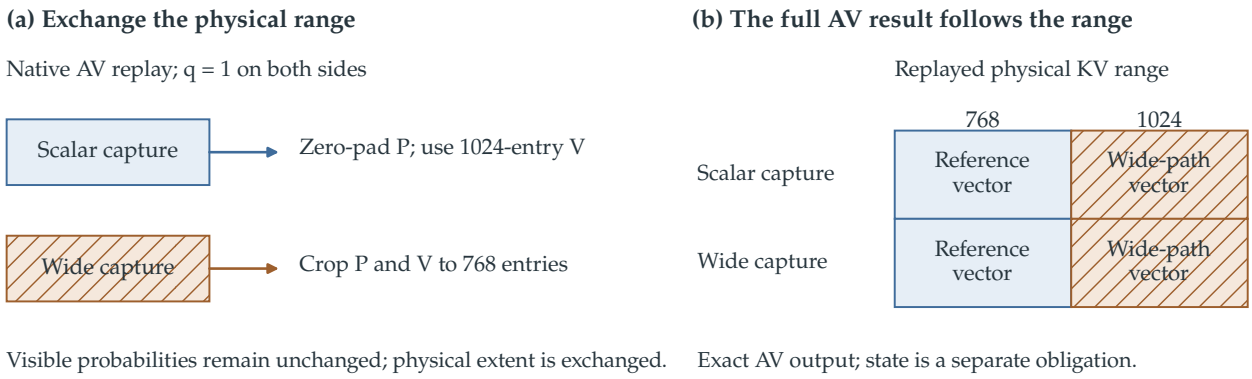



**Figure 2:** A physical-range intervention changes the attention–value result. The captured CUDA comparison fixes a one-query consumer and tests 768- and 1024-entry physical ranges. Cropping and padding reproduce the respective reference outputs while retaining the visible operands. The observations concern the complete output of this operator; they do not measure KV-state equivalence or service time. The separate Vulkan study changes layout and dispatch together with range and is therefore a joint repair, not an isolated range intervention. Data and intervention identities are in the accompanying figure manifest.

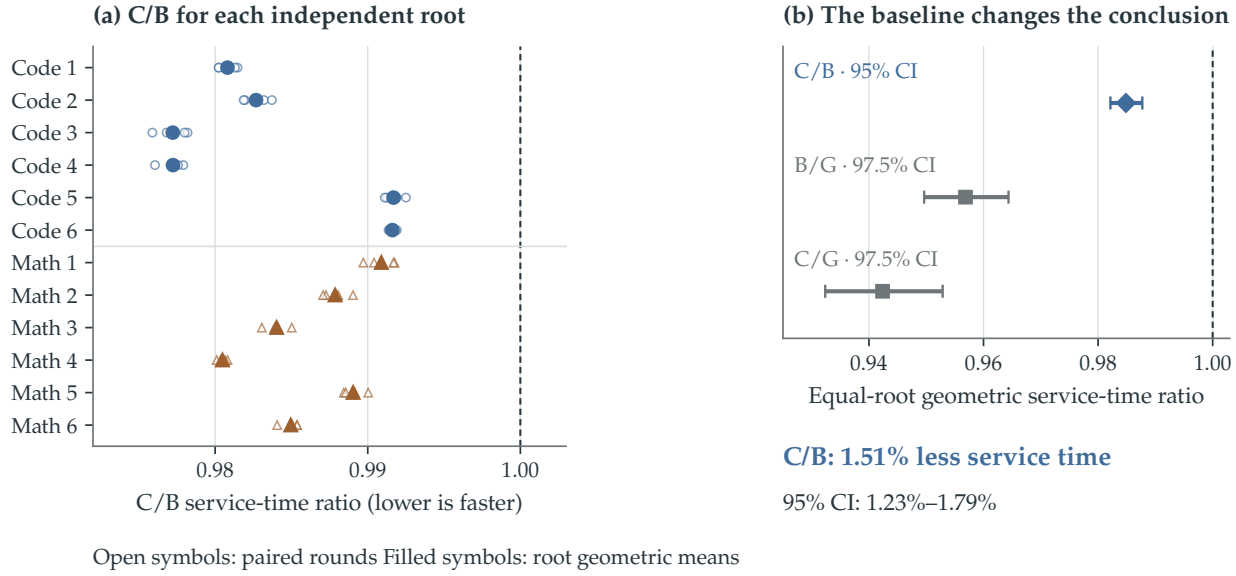



**Figure 3:** The incremental service benefit beyond a strong same-contract control. A5000, Qwen3-32B-Q4_K_M, target-only generation, fixed whole-graph cuts. Root points average paired log ratios over four rounds. The primary C/B summary weights 12 roots equally and uses a 95% paired-root interval stratified by coding and mathematics roots. Secondary B/G and C/G summaries use 97.5% intervals. B shares activation quantization and invokes scalar backend projections; C schedules those scalar computations as independent channels. G is the weaker graph-level scalar loop. The resident timer includes reset and output ownership but excludes loading and diagnostic capture.

of the advantage over G is already obtained by moving the scalar loop into the backend and sharing quantization. C/B isolates the remaining organization change. Across the 48 requests, B and C take 364.832 and 358.925 s in total; their pooled ratio, 0.983809, differs from Eq. 4. Prefill falls from 143.927 to 137.984 s, while decode is effectively unchanged at 216.663 and 216.690 s. This localization agrees with the opportunity to group known prompt rows. The result establishes a measurable compatibility-preserving increment in this deployment, without assigning draft-model benefits to it.

### 5.3. Packing and local cuts have different opportunities

The Q8 factorials fix the draft policy and cross cut placement with projection organization (Fig. 4). Packing reduces service by 1.64% with SIMPLE and 1.56% with LOCAL at 8B, and by 0.99% and 0.80% at 14B. Their backend-loop4 controls already share activation

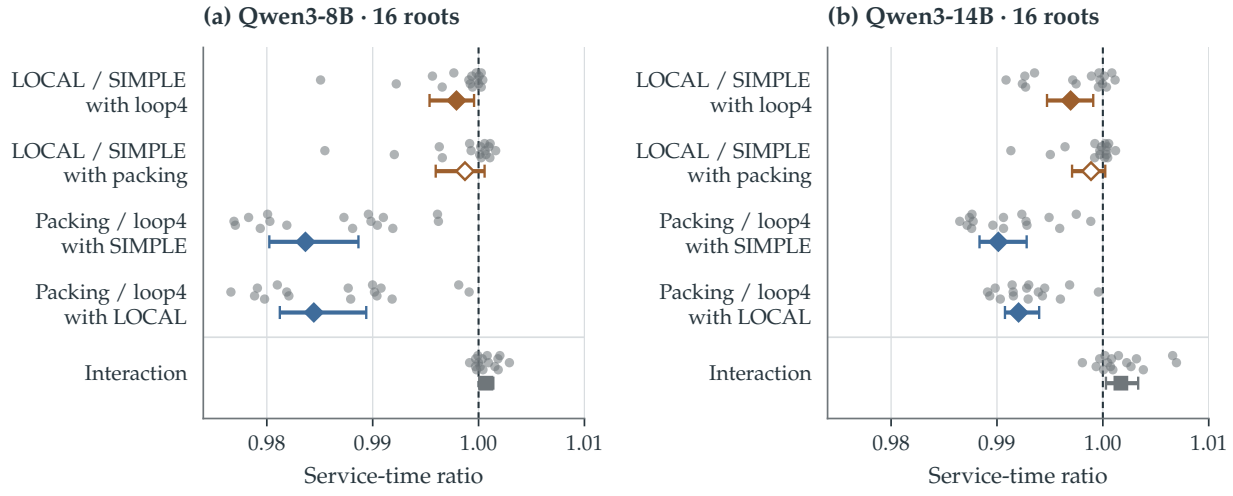



**Figure 4:** Separating the two execution changes. Balanced A5000 Q8 factorials fix draft prefill microbatch four and proposal count three. Faint points show individual root ratios; summary markers show ratios of total resident-service time with 99% paired-root intervals from the five-comparison family. Projection contrasts hold cut placement fixed; cut contrasts hold projection fixed. The interaction summary is an equal-root geometric ratio of ratios. The two models share 16 task roots and are separate measurements. Intervals spanning one are visibly retained.

quantization, so these contrasts measure the remaining packed-group increment.

Local cuts reduce service by 0.21% and 0.30% at fixed loop4. At fixed packing, the reductions are 0.13% and 0.11%, with both intervals containing zero reduction. The equal-root interactions are 1.000696 and 1.001703, indicating slight attenuation when both components are applied. At $k = 3$, target verification has at most four queries, so it does not enter the multi-group Q8 packing route. Execution records place that route in prefill. The components consequently address different opportunities even within one request.

## 5.4. Boundary geometry explains the local-cut results

In the natural 8B factorial, 20 of 2,224 logical verification cycles require an additional whole-graph split. SIMPLE uses 2,244 graphs and LOCAL 2,224 for the same target rows: just 0.899% of cycles present that opportunity. The controlled B570 appends make the opportunity explicit (Fig. 5). A crossing starts one row before a physical boundary and forces the whole-graph split $1 + (q - 1)$, for $q = 2, 4, 32$. Across capacities 1024 and 4096, local execution takes 20.34–44.68% less append time in these cells. Matched inside controls take 0.43–3.74% more time. Three paired observations per cell show repeat variation, not a workload confidence interval.

Crossing alone is not sufficient. Earlier $q = 32$ cells with a $16 + 16$ whole-graph split include cases favoring SIMPLE. In the natural multi-turn pilot, the four paired episode ratios are 1.0997, 1.0024, 1.0886 and 0.9888 (Table 2). Across the eight timed episodes, LOCAL takes 37.921 s versus 36.385 s for SIMPLE, a descriptive ratio of 1.04220. The pilot therefore provides no observed local-cut service improvement; two development conversations cannot establish a population ranking.

**Table 2**
Natural multi-turn pilot: LOCAL/SIMPLE resident-service ratios. Packing and the request protocol are fixed. Rows are two rounds of each development episode, not four independent tasks. No confidence interval is inferred.

| Episode | Round 1 | Round 2 |
|---|---|---|
| Development conversation A | 1.0997 | 1.0024 |
| Development conversation B | 1.0886 | 0.9888 |
| Pooled ratio | | 1.04220 |

The pilot's complete calls fall from 1,072 to 1,070. A separate source-derived plan for one proposed 89-row supplied-history append finds four versus three decoder calls but the same 23 ordered projection tiles. Fewer outer calls can therefore leave the principal arithmetic work unchanged. These observations identify limits on the natural-request opportunity: local placement avoids some graph fragmentation, while view construction, attention launches and concatenation remain real costs.

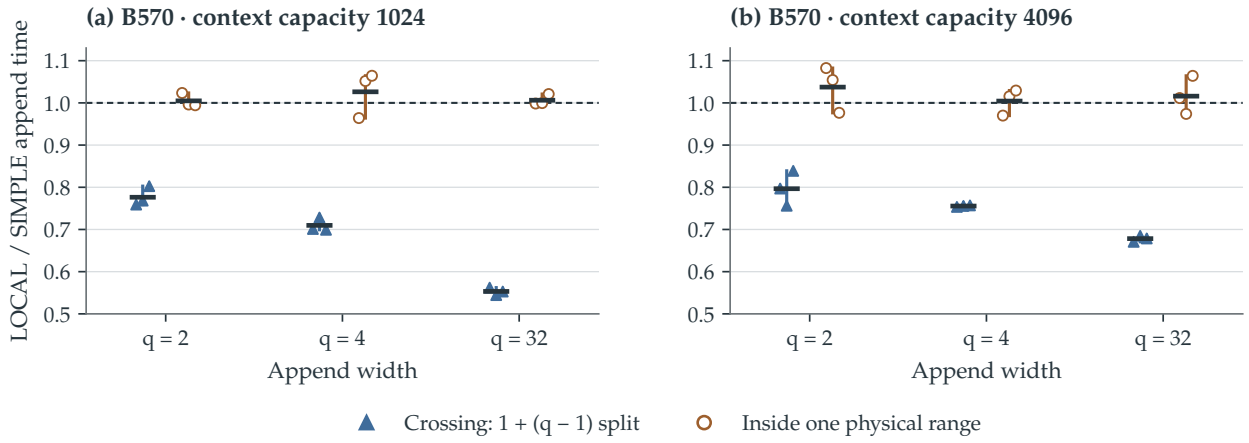



**Figure 5:** Local cuts help when they retain enough surrounding work. The opportunity count on A5000 is shown separately from B570 operation timings. B570 points compare LOCAL/SIMPLE append time at fixed $q$ and capacity; crossing triangles and inside circles use distinct shapes and fills. Ranges are the minimum and maximum of three paired repeats. The prepared-state timer includes binding, decode, synchronization and requested-output copying, and excludes state preparation. Capacity and prefix region change together between the two capacity panels. Earlier $16 + 16$ crossings, including reversals, are retained in the supplement.

### 5.5. Application gains are larger than the method increment

The complete Q8 speculative applications improve on all four target-only controls (Fig. 6). Against the fastest measured fixed M, LOCAL-packed D4 reduces resident service by 17.21% for Qwen3-8B, 25.54% for Qwen3-14B, and 11.52% for Llama-8B. Their adjusted 99% intervals are [11.17%, 24.61%], [18.21%, 34.89%], and [4.33%, 20.95%], respectively. These comparisons include the draft and the complete verification application. Against SIMPLE-loop4 with the same D4 policy, the corresponding increments are 2.01%, 1.08% and 1.56%.

Initialization can change the decision. For the 14B request burst, worker time including loading, integrity checks and primers falls from 662.546 s for W-P-M to 594.559 s for LOCAL-packed D4. Llama's resident requests improve, yet its short worker workload favors target-only execution: the fastest M worker takes 355.730 s versus 368.822 s for LOCAL-packed D4. The earlier 32B panel instead favors ordinary grouped generation; grouped D4 takes about 14% more resident time. A B570 pilot likewise favors target-only generation despite fewer target graphs in the speculative path. The supplement retains these complete cost accounts.

Wider execution also needs a workload distinction. A Windows RTX equal-work diagnostic makes one $q = 8$ call 3.94–11.91% cheaper than two $q = 4$ calls from the same state with the same eight known tokens and requested outputs. In live generation, increasing proposals from $k = 3$ to $k = 7$ raises service time by 13.58–14.59% across the four methods. For the same 11,472 emitted tokens, verification cycles fall from 3,504 to 2,296, but verified rows rise from 14,016 to 18,368 and draft proposals from 10,512 to 16,072. Avoiding launches does not repay the extra unconsumed work.

### 5.6. A modern backend is a different engineering choice

A recent upstream llama.cpp few-row implementation [13] provides a useful alternative. On one captured Qwen3-8B Q8 input with $K = N = 4096$ and $q = 8$, the ratio of upstream U to the extracted legacy packed Q8 operator B is 0.612924, with a 95% paired-repeat interval [0.607906, 0.618665]. The 38.7% reduction belongs to that operator comparison. U uses a different quantization and projection contract: 32,766 of 32,768 output bit patterns differ from the legacy reference. The preceding attempt to qualify U against the earlier strict CPU-round-to-nearest oracle for the upstream path failed and produced no timing under that oracle. The later comparison declares U's own contract instead.

This result favors considering a numerical migration when exact compatibility is unnecessary. It does not measure a complete default upstream backend: that backend's qualification and paired service timing have not been run. Two legacy-compatible extracted candidates also fail to improve the strong control, with complete-operator ratios 1.1264 and 1.0261 in their separate sessions. The available measurements compare operator choices; they leave the complete-backend service comparison open.

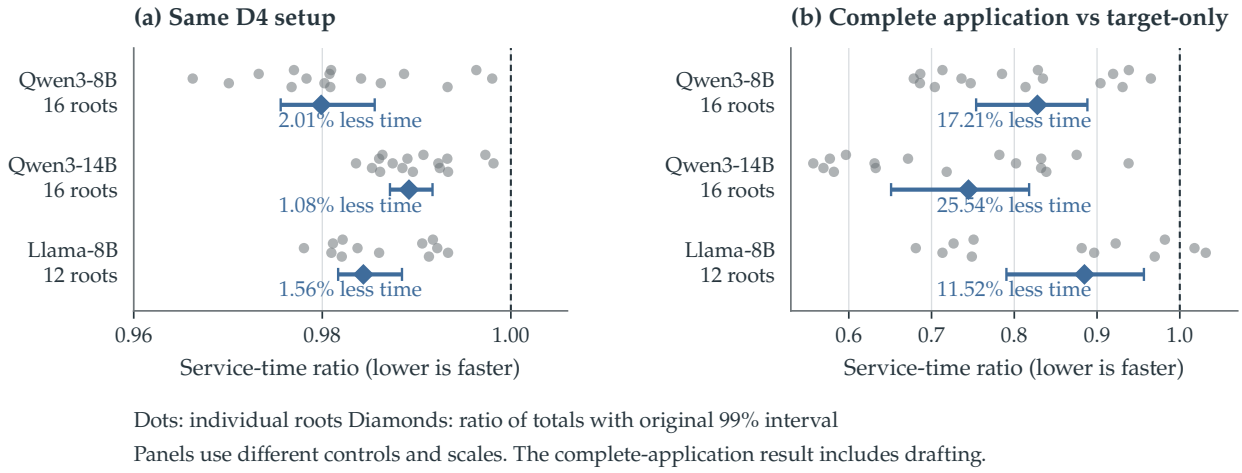



**Figure 6:** Application value and execution increment answer different questions. Panel (a) compares LOCAL-packed D4 with SIMPLE-loop4 D4 under the same draft policy; panel (b) compares it with the fastest measured fixed target-only control. Points retain root heterogeneity; summaries are ratios of total resident-service time with 99% paired-root intervals. Qwen panels use the same 16 roots; Llama uses 12 separately selected roots. All fixed target-only comparisons are retained in the supplement, so choosing the fastest measured M does not replace the prespecified control family. The larger application reductions include speculative decoding.

## 6. Related work

*Matching an existing numerical reference.* Equality of mathematical operations does not determine their floating-point execution. Yang et al. reconstruct the bitwise behavior of cuBLAS kernels, describe the factors that determine reduction order, and enforce those factors in generated kernels [3]. Their separation of arithmetic structure from scheduling already permits changes to output tiling and pipelining when the per-output computation is preserved. Their attention treatment is theoretical; the implemented reconstruction and verification focus on reduction and matrix-multiplication kernels. Our work addresses a different composition problem: executing quantized transformer queries against an existing reference while retaining the physical KV extent and persistent state that the surrounding decoder expects. The method couples compatible arithmetic groups with local cuts in the query sequence. We compose these constraints at decoder boundaries and test the resulting implementation under the stated deployment conditions.

*Batch invariance and deterministic scheduling.* He and Thinking Machines Lab identify batch-dependent reduction order as a source of inference nondeterminism and construct batch-invariant normalization, matrix multiplication, and attention [1]. Their attention discussion also explains how the boundary between cached and newly computed KV entries can alter arithmetic grouping. Thus, fixed reductions and the numerical significance of KV boundaries are established ideas. Our physical-range analysis asks which extent the selected reference actually uses, including initialized entries outside the logical causal prefix, and which query groups can share that extent without changing their arithmetic.

CoRun obtains deterministic inference through isolated prefill, fixed-shape batched decode, and position-invariant kernels [2]. It coordinates execution shape, attention, collective reductions, and request-specific sampling state. This provides an alternative to making every kernel batch invariant. The relevant distinction is the chosen reference: a system can select a new internally consistent execution policy, whereas compatibility with an existing reference constrains which of those choices remain available. Our local cuts preserve such a reference at the points where combining queries would change it, while allowing compatible work to remain grouped.

*Output and persistent-state obligations.* Vosti specifies deterministic inference at the system level and verifies engine invariants together with kernel contracts [14]. Its central KV invariant ties every live or retained reusable cache entry to the value obtained from its logical token prefix. Layout, write isolation, kernel selection, and sampler state participate in the argument. This prior work establishes why determinism cannot be judged from emitted tokens alone. Our consumer obligations serve a related purpose for an existing implementation: they identify the output bytes and successor state that must agree before a changed execution can be reused downstream. We give a conditional composition argument and evaluate selected obligations, rather than claiming Vosti's mechanized system guarantee.

LLM-42 takes another route: a dynamically batched fast path generates candidates, and a fixed-shape verifier commits a consistent prefix or rolls back [4]. It replaces fast-path KV entries with verifier-produced entries even when token verification succeeds. Our construction instead organizes compatible execution before committing its results. This trades some scheduling freedom for avoiding verification and repair as the normal execution strategy. The two approaches therefore place their compatibility work at different points in the inference pipeline.

*Quantized kernels and inference optimization.* The implementation inherits deterministic quantized operations and four-column execution from `llama_reproducible` [6]. Modern `llama.cpp` also provides a few-row quantized matrix-multiplication path using integer tensor-core operations, asynchronous copies, and reuse across column groups [13]. We study the additional constraints that determine where compatible query groups can use packing and weight reuse. FlashAttention reduces memory traffic through tiled attention [15], while FlashInfer supports heterogeneous KV layouts and load-balanced attention execution [16]. Their optimization objectives do not require reproducing the bytes of the particular reference studied here. Our experiments separate comparisons within that contract from comparisons with a newer numerical path, so that the cost of retaining the reference remains visible.

*Speculative decoding.* Speculative decoding and sampling amortize target-model work by verifying several draft tokens together [17, 18]. Their distribution-preservation arguments address a different requirement from retaining all outputs and persistent state of a fixed numerical execution. Liu et al. adapt verification effort to candidate uncertainty [19], improving how speculative work is selected. Our method concerns how selected queries execute once a numerical reference has been fixed. We report complete speculative-application comparisons separately from four-arm experiments that hold the draft policy fixed and isolate packing, local cuts, and their interaction.

## 7. Discussion and limitations

### 7.1. When retaining a reference is worth the cost

The fixed reference is an application constraint, not a universal performance objective. It can isolate scheduling changes during regression testing, permit exact replay from stored state, or support a component migration while its consumer remains unchanged. In such settings, a small execution improvement can be useful because the optimized path preserves the expected numerical interface. The 32B result shows that this option has measurable value beyond a strong implementation control. It does not establish a production return on maintenance effort, energy savings, or a general preference for the legacy backend.

A deployment free to change its numerical contract should evaluate that option directly. The modern extracted operator illustrates the possible benefit. A fair system decision would compare complete, appropriately qualified backends on the intended workload and include the cost of migration or compatibility. Our current evidence contains the legacy-compatible service comparison and the modern operator comparison, with different scopes; the missing full-backend comparison remains an open engineering decision rather than an assumed win for either choice.

### 7.2. What transfers beyond this implementation

The transferable object is the construction procedure: derive the physical and arithmetic constraints of a chosen reference, place the required cuts at the operations that depend on them, and check the consumer's outputs and successor state. It directs implementation and testing even when a backend uses different widths or cache layouts.

The implementation remains specialized. Its two projection widths and CUDA/Vulkan paths demonstrate instances of the construction, not automatic transfer to arbitrary quantizers, paged or sliding caches, mixed-device partitions, Flash attention, concurrent sequences, or stochastic sampling. Adding such a path requires deriving its numerical descriptor, discharging its state obligations and qualifying its actual dispatch. The conditional composition argument helps identify those obligations; it does not replace that work. Likewise, existing batching and packing techniques supply components, so the contribution should be judged by the combined execution organization and the evidence explaining its costs.

### 7.3. Why the negative results matter

The results expose three distinct constraints on performance. A local cut has little value when a request seldom crosses the relevant physical boundary. Even at a crossing, preserving a wider graph may leave the same arithmetic tiles or incur enough attention overhead to lose. Finally, a more efficient fixed operation can still make an application slower when it induces additional draft and target work. These mechanisms explain the small factorial cut effect, the multi-turn pilot and the wider-speculation reversal without requiring a universal threshold for useful speedup.

The evidence is also limited by its sampling. Several studies reuse development roots, and the Qwen scales reuse input identities. Confidence intervals preserve root clustering and describe the stated sessions; they do not measure cross-machine, cross-day or population-wide variation. Structural cells and the two-episode pilot are descriptive. Workload identities, orders, clocks

and uncertainty types therefore remain explicit in the figures and supplement.

### 7.4. Reproducibility and implementation maturity

Public timing rows support independent statistical reconstruction, and the source maps identify the code and public diagnostics behind each result. Some native records are company-restricted, and some temporary intermediates were pruned. Public summaries cannot substitute for unavailable original tensors. The present reconstruction checks published statistics and implementations without rerunning native qualification. The final archival deposit and any permitted restricted-access procedure require author confirmation.

The current command-line implementation also has a narrower recovery boundary than a persistent inference service. Target-group errors poison the context, while other failures terminate the worker. A reusable service library would need transaction-wide invalidation and explicit admission for the unsupported graph cases. These requirements concern deployment maturity; the measured service timer remains the complete resident request path in the stated worker.

## 8. Conclusion

Fixed numerical references leave useful parallel work available in quantized LLM inference. Separating physical KV ranges from arithmetic groups allows the required attention boundaries to remain local while compatible projections and surrounding computation remain grouped. Output ownership and state commit rules make that execution usable by downstream consumers. The evaluated implementations provide a small but measurable service increment beyond a strong same-contract control, while component and application experiments explain why local cuts, wider verification and complete speculation have different outcomes. The method offers a compatibility-preserving execution option whose value can be assessed against both its own costs and the alternative of adopting a new numerical backend.

## Data and code availability

Implementation sources, public timing rows, plotting inputs and CPU reconstruction scripts are identified by fixed revisions in the supplementary material and accompanying source manifest [5]. The manuscript package includes the public inputs required to regenerate its numerical figures. Some underlying native logs and tensor records are company-restricted; selected temporary intermediates were pruned and are no longer available. No general access to those materials is promised. The archival data repository, the final reason and scope of restrictions, and any authorized access procedure remain to be confirmed by the author before submission.

## Declaration of generative AI and AI-assisted technologies

During preparation of this author-review draft, ChatGPT (OpenAI, GPT-6 Astra Pro, accessed 13 November 2026) assisted manuscript restructuring and English editing, literature and code checks, CPU statistical reconstruction, plotting code, a conceptual execution diagram, and LaTeX build and layout review. Numerical figures were generated from the identified data with reproducible scripts. AI assistance in the prior research-code workflow is reported in the project records; the author must verify the complete historical tool and model information. Final author review of all scientific content and acceptance of responsibility are pending. This declaration is a review draft, not a statement that final author approval has occurred.

## Author declarations awaiting confirmation

The author must confirm the complete affiliation and postal address, CRediT contribution statement, funding information, competing-interest declaration, acknowledgments, permissions and final data-access terms before submission. No funding source, conflict status or contributor role is inferred in this draft.

## References


[1] Horace He and Thinking Machines Lab. Defeating nondeterminism in LLM inference. *Thinking Machines Lab: Connectionism*, September 2025. doi: 10.64434/tml.20250910. URL `https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/`.

[2] Shiju Zhao, Jiacheng Yang, Qihang Chen, Junhao Hu, Jiaqi Zheng, Guihai Chen, and Xusheng Chen. CoRun: Padding is simple and efficient for deterministic LLM inference. arXiv:2608.14376v1, 2026. URL `https://arxiv.org/abs/2608.14376v1`.

[3] Ziteng Yang, Nicholas J. Riasanovsky, Warren Deng, and Vivek Sarkar. Taming bitwise behavior in GPU kernels with tensor core: Black-box reconstruction, compiler enforcement, and static verification. arXiv:2609.11356v1, 2026. URL `https://arxiv.org/abs/2609.11356v1`.

[4] Raja Gond, Aditya K. Kamath, Ramachandran Ramjee, and Ashish Panwar. LLM-42: Enabling determinism in LLM inference with verified speculation. arXiv:2601.17768v2, 2026. URL `https://arxiv.org/abs/2601.17768v2`.

[5] leecdiang. Implementation and companion evidence for fixed-reference parallel inference. GitHub repository, 2026. URL `https://github.com/leecdiang/HeteroSpec`. Experiment commits, evidence locations, and access scope are listed in the supplementary material.

[6] Andrew Miller. llama_reproducible: Deterministic quantized inference implementation. GitHub repository, 2026. URL `https://github.com/amiller/llama_reproducible/tree/f0dafdbd9dba40bd0e596f74f3da5f592900fca4`. Commit f0dafdbd9dba40bd0e596f74f3da5f592900fca4; accessed November 13, 2026.

[7] An Yang et al. Qwen3 technical report. arXiv:2505.09388v1, 2025. URL https://arxiv.org/abs/2505.09388v1.

[8] Aaron Grattafiori et al. The Llama 3 herd of models. arXiv:2407.21783v3, 2024. URL https://arxiv.org/abs/2407.21783v3.

[9] Loubna Ben Allal et al. SmolLM2: When smol goes big—data-centric training of a small language model. arXiv:2502.02737v1, 2025. URL https://arxiv.org/abs/2502.02737v1.

[10] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. Measuring coding challenge competence with APPS. arXiv:2105.09938v3, 2021. URL https://arxiv.org/abs/2105.09938v3.

[11] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. arXiv:2103.03874v2, 2021. URL https://arxiv.org/abs/2103.03874v2.

[12] Andreas Köpf, Yannic Kilcher, Dimitri von Rütte, Sotiris Anagnostidis, Zhi-Rui Tam, Keith Stevens, Abdullah Barhoum, Nguyen Minh Duc, Oliver Stanley, Richárd Nagyfi, Shahul ES, Sameer Suri, David Glushkov, Arnav Dantuluri, Andrew Maguire, Christoph Schuhmann, Huu Nguyen, and Alexander Mattick. OpenAssistant conversations—democratizing large language model alignment. arXiv:2304.07327v2, 2023. URL https://arxiv.org/abs/2304.07327v2.

[13] llama.cpp contributors. Few-row quantized matrix multiplication in llama.cpp. Source code, ggml/src/ggml-cuda/mmq-few.cu, 2026. URL https://github.com/ggml-org/llama.cpp/blob/0fb8f157702702f33f4e018de8701b2569a1bc28/ggml/src/ggml-cuda/mmq-few.cu. Commit 0fb8f157702702f33f4e018de8701b2569a1bc28; accessed November 13, 2026.

[14] Jianxing Qin, Alexander Du, Danfeng Zhang, Matthew Lentz, and Danyang Zhuo. Vosti: Specifying, implementing, and verifying deterministic LLM inference. arXiv:2609.38981v1, 2026. URL https://arxiv.org/abs/2609.38981v1.

[15] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. arXiv:2205.14135v2, 2022. URL https://arxiv.org/abs/2205.14135v2.

[16] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. FlashInfer: Efficient and customizable attention engine for LLM inference serving. arXiv:2501.01005v2, 2025. URL https://arxiv.org/abs/2501.01005v2.

[17] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 19274–19286. PMLR, 2023. URL https://proceedings.mlr.press/v202/leviathan23a.html.

[18] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. arXiv:2302.01318v1, 2023. URL https://arxiv.org/abs/2302.01318v1.

[19] Siran Liu, Yang Ye, Qianchao Zhu, Zane Cao, and Yongchao He. HeteroSpec: Leveraging contextual heterogeneity for efficient speculative decoding. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12930–12947. Association for Computational Linguistics, July 2026. doi: 10.18653/v1/2026.acl-long.589. URL https://aclanthology.org/2026.acl-long.589/.